



Gemini ADK 마스터클래스

단일 에이전트 설계부터 멀티 에이전트 시스템 구축까지

에이전트 생성 과정



1. 탄생 (Genesis)

환경 설정 및
단일 에이전트의 구조

2. 확장 (Capabilities)

도구의 결합과
기억력 확보

3. 사회 (Orchestration)

워크플로우 제어와
멀티 에이전트 협업

4. 연결 (World Integration)

MCP를 통한 외부
클라우드 생태계 통합

5. 통제 (Mastery)

콜백 메커니즘을 이용한
강력한 가드레일

개발자를 위한 코드 중심 오픈소스 프레임워크



No-Code Agent의 한계

정해진 틀 안에서만 동작하여 오케스트레이션 및 세밀한 제어가 제한됨



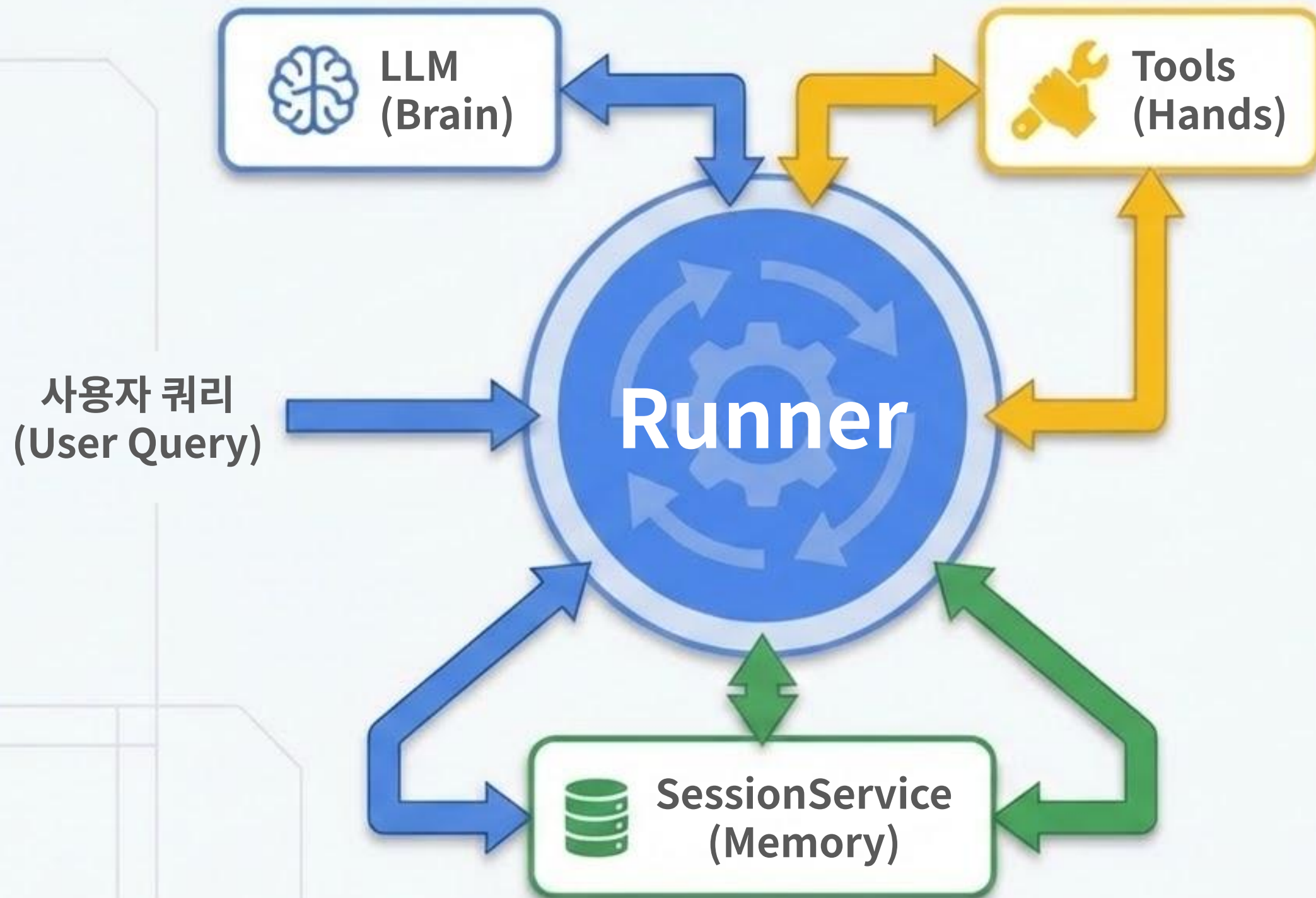
Python 코드 중심 오케스트레이션 제공

개발자가 에이전트의 동작과 협업 과정을 직접 정의하여 무한한 확장성과 세밀한 제어 권한을 확보

Apache 2.0 라이선스 기반으로 한 완전한 테스트 가능 구조

보안원칙 : 환경 설정 파일을 통한 안전한 자격 증명 관리

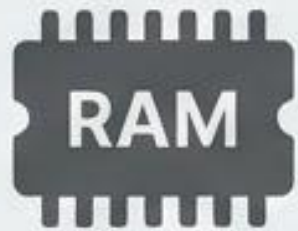
시스템의 중심 : Runner



Runner 핵심 기능 (Core Functions)

- ✓ **사용자 쿼리**를 처리하는 메인 진입점이자, 에이전트의 실행 루프를 조율하는 핵심 오케스트레이터
- ✓ **이벤트 루프 관리** : 에이전트 로직, 대규모 언어 모델(LLM), 그리고 외부 도구 호출을 비동기적으로 중재
- ✓ **상태 동기화** : 쿼리 처리 과정에서 생성된 모든 상호작용 이벤트를 추적하고 상위 시스템으로 안전하게 전달

대화 상태의 지속성 관리 : SessionService



InMemorySessionService

빠른 테스트와 로컬 개발용.

데이터는 애플리케이션 메모리에 일시적(Ephemeral)으로 저장되며 재시작 시 초기화



DatabaseSessionService

자체 관리형 지속성 저장소

PostgreSQL, MySQL 등에 테이블 형태로 대화 상태가 영구적으로 보관



VertexAiSessionService

프로덕션 수준의 확장성

Google Cloud에 배포 시 활용 가능한 강력한 관리형 세션 서비스



상태 스코프 제어(State Scope Control) : app;, user;, temp; 접두사를 통해 데이터의 유효 범위를 가장 정밀하게 관리합니다.

다층적 맥락 구조 (Layered Context)

Invocation (단일 요청)



최상위 컨테이너, 단일 사용자 요청 처리에 필요한 핵심 로직과 Session, Services, Identifier 등 모든 필수 컨텍스트 제공.

State (현재 스레드)



tool_context.state 기반 임시 데이터 저장소.
도구(Tool) 및 하위 에이전트 간 임시 결과 공유.
상태 스코프 접두사 지원 : app;, user;, temp;



Session & Memory (전체 기록/지식)

Session : 진행 중인 상호작용의 전체 타임 라인 추적.

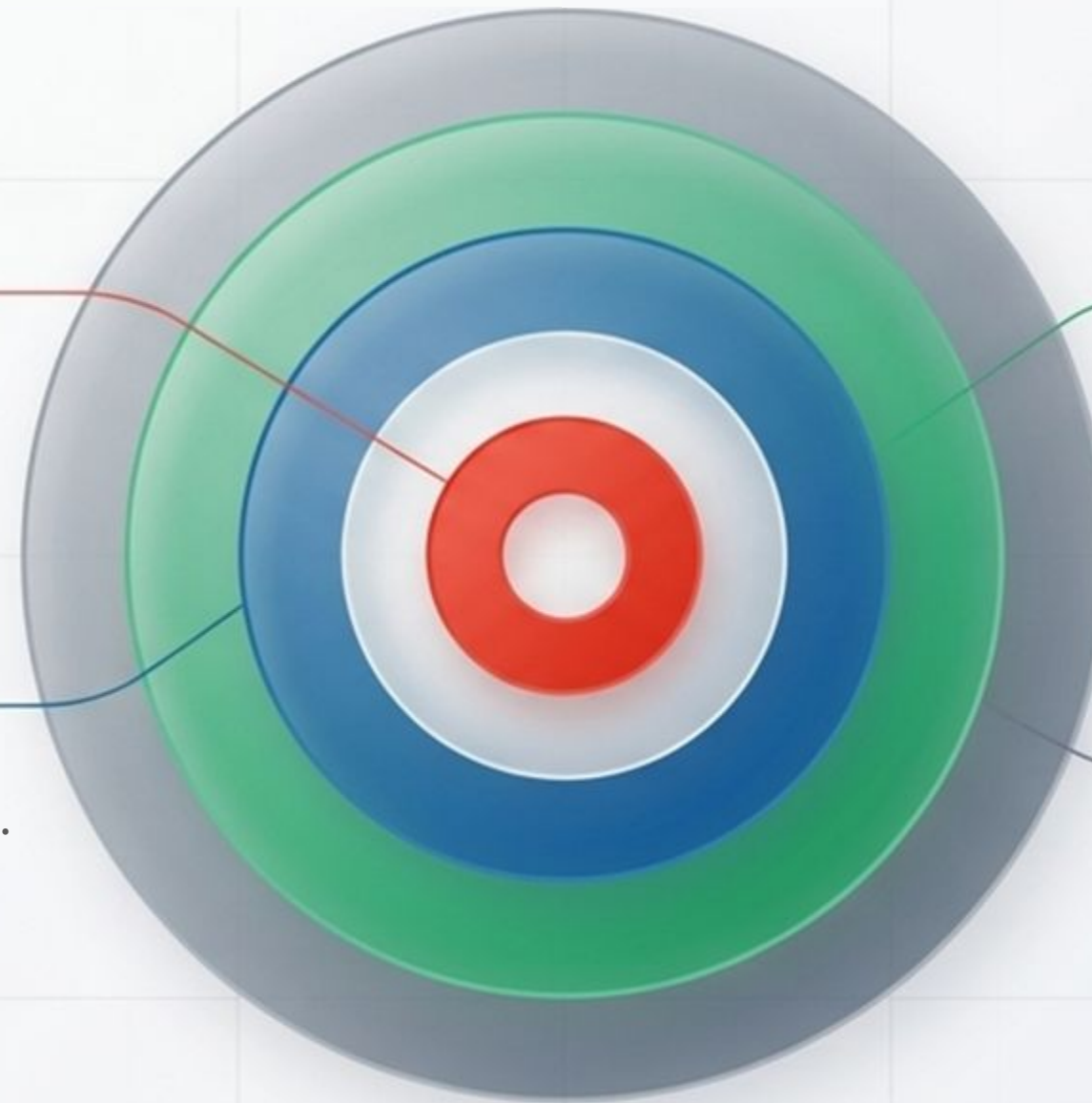
Memory : 과거 세션 기반 지식 조회 및 외부 데이터와 연계 가능 (search_memory)



Artifacts (바이너리/파일)

텍스트의 한계를 넘는 영속화 계층.

save_artifact / load_artifact 를 통해 파일, 이미지, 오디오 등 이름과 버전이 지정된 바이너리 데이터 관리



컨텍스트는 단순한 텍스트 묶음이 아닙니다.
에이전트가 대화의 맥락을 잃지 않고 작업을 수행하게 만드는 입체적인 배경 지식망입니다.

에이전트(Agent) vs 도구(Tool) 의 본질적인 차이

	에이전트 (Agent)	도구 (Tool)
		
자율성 (Autonomy)	환경을 인지하고 언제 어떤 행동을 할지 스스로 판단하고 결정합니다.	스스로 의사결정을 내릴 수 없으며, 에이전트가 호출할 때만 수동적으로 동작합니다.
역할 (Role)	대화의 전반적인 흐름을 관리하고, 사용자에게 맞춘 워크플로우를 오케스트레이션 합니다.	에이전트가 외부 세계와 상호작용하고 구체적인 단일 작업을 수행하기 위한 수단입니다.
구현 모델 (Implementation)	프레임워크 내부의 BaseAgent, LlmAgent 클래스 기반으로 인스턴스화 됩니다.	순수 Python 함수, Langchain, CrewAI 등 서드파티 라이브러리를 래핑한 모듈 형태입니다.

통제권의 위임 : Sub-agent vs Agent-as-a-Tool



Agent-as-a-Tool (결과 반환)

상위 에이전트가 제어권을 계속 유지합니다. 특정 작업을 전문 에이전트에게 위임한 후, 그 결과값을 다시 반환 받아 최종 응답을 상위 에이전트가 직접 구성합니다. (예 : 정보 검색 후 요약 보고서 작성)

Sub-agent (통제권 완전 이전)

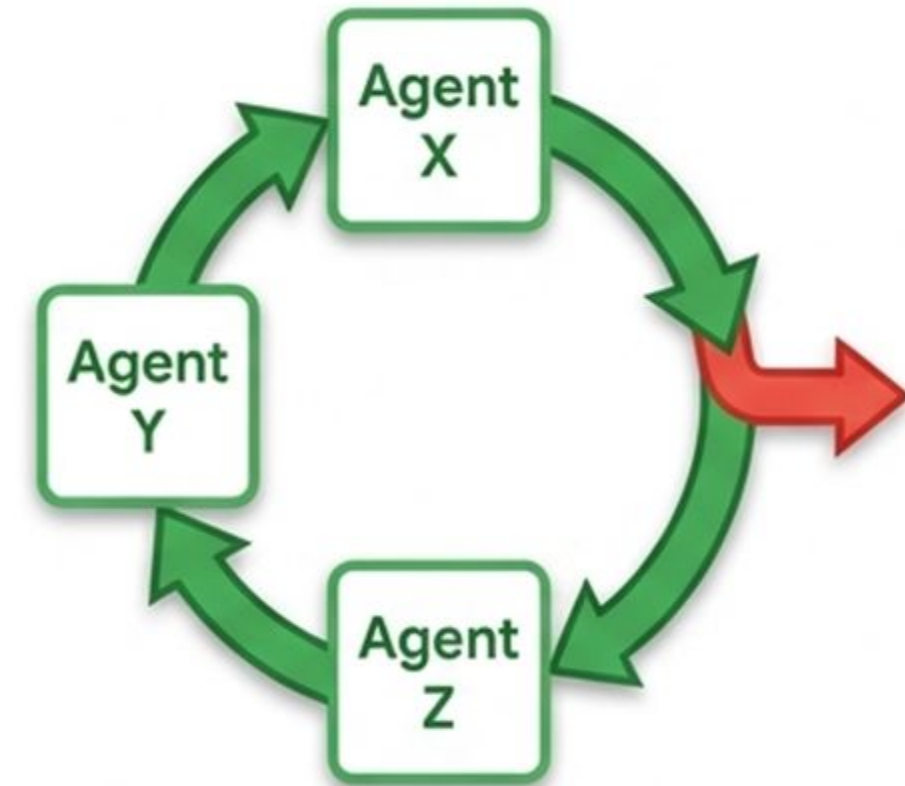
자동 위임(Automatic Delegation) 기술입니다. 대화 라우터처럼 동작하여, 해당 전문성을 가진 하위 에이전트에게 사용자 응답에 대한 책임 자체를 완전히 넘깁니다.

결정론적 워크플로우 제어 : 순차와 반복



SequentialAgent (순차 구조)

동일한 InvocationContext를 하위 에이전트 간에 정해진 순서대로 전달합니다.
'조립 라인'처럼 이전 단계 에이전트의 상태(State)가 다음 단계의 입력이 되는 명확한 의존성 처리를 요구하는 파이프라인에 완벽합니다.



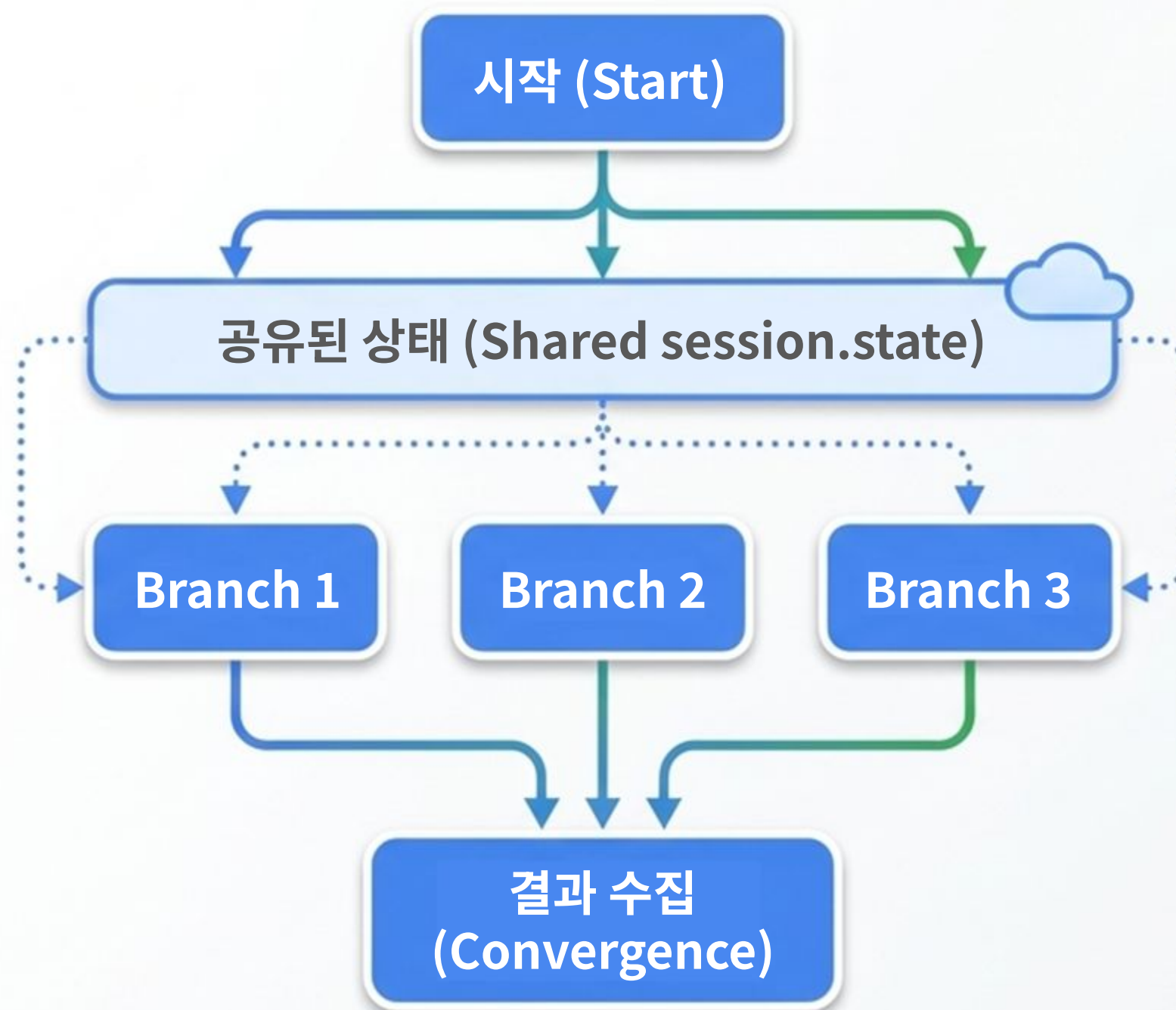
LoopAgent (반복 구조)

하위 에이전트들을 설정된 조건에 따라 순환 반복 실행합니다. 코드 리뷰나 아이디어 세분화 등 결과물을 점진적으로 개선할 때 쓰입니다.
max_iterations 도달 또는 actions.escalate=True 신호 발생 시 루프를 안전하게 종료하고 탈출합니다.

동시성 처리를 통한 성능 극대화 : ParallelAgent

성능 극대화

여러 하위 에이전트에게
동시에(Concurrently) 작업을
지시하여, 순차 처리 대비 전체
실행 시간을 단축할 수
있습니다.
리소스 집약적인 데이터
수집에 이상적입니다.



독립적 경로와 공유된 상태

각 에이전트는
InvocationContext.branch를
통해 분리된 실행 경로를
보장받지만, 동일한 session.state
풀에 접근합니다.

데이터 덮어쓰기 충돌을 막기 위해
결과 저장 시 고유한 키(Key)
할당이 필수적입니다.

실무 적용 예 : 여행 계획 시스템



단일 프롬프트의 한계 극복

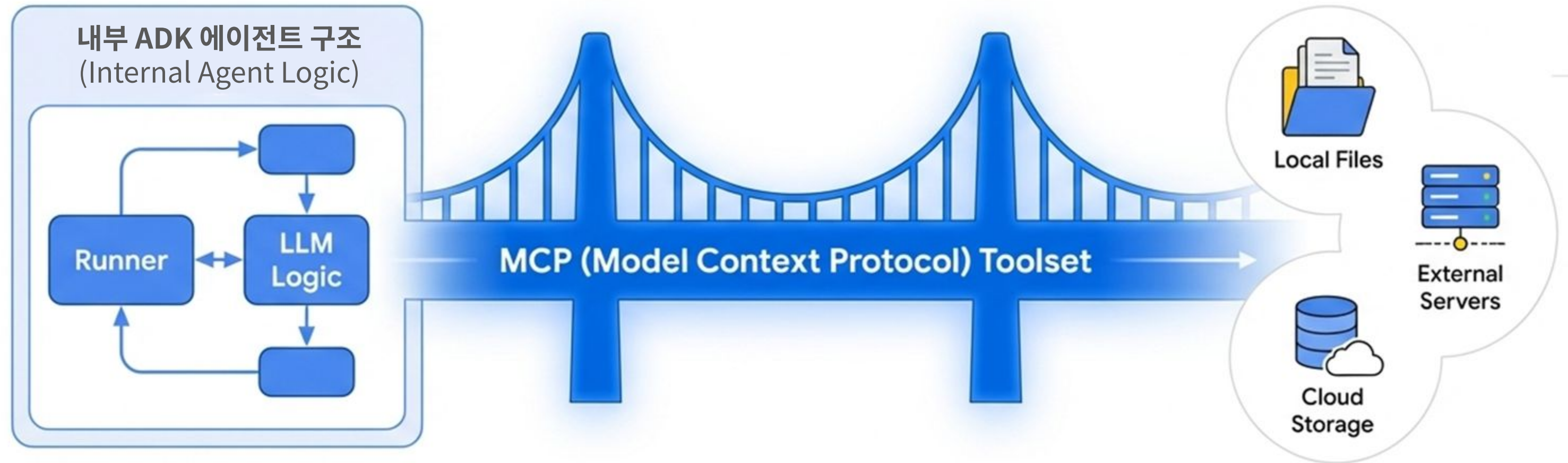
시스템 지침 분할 :

거대한 하나의 프롬프트에 의존하는 대신 역할을 분리하여 각 에이전트에게 고도로 전문화된 컨텍스트와 도구를 부여합니다.

정확성 극대화 :

이러한 에이전트 간 오케스트레이션 구조는 복잡하고 모호한 사용자 의도를 정확하고 풍부한 최종 결과물로 변환하는 멀티 에이전트 시스템의 효과를 잘 보여줍니다.

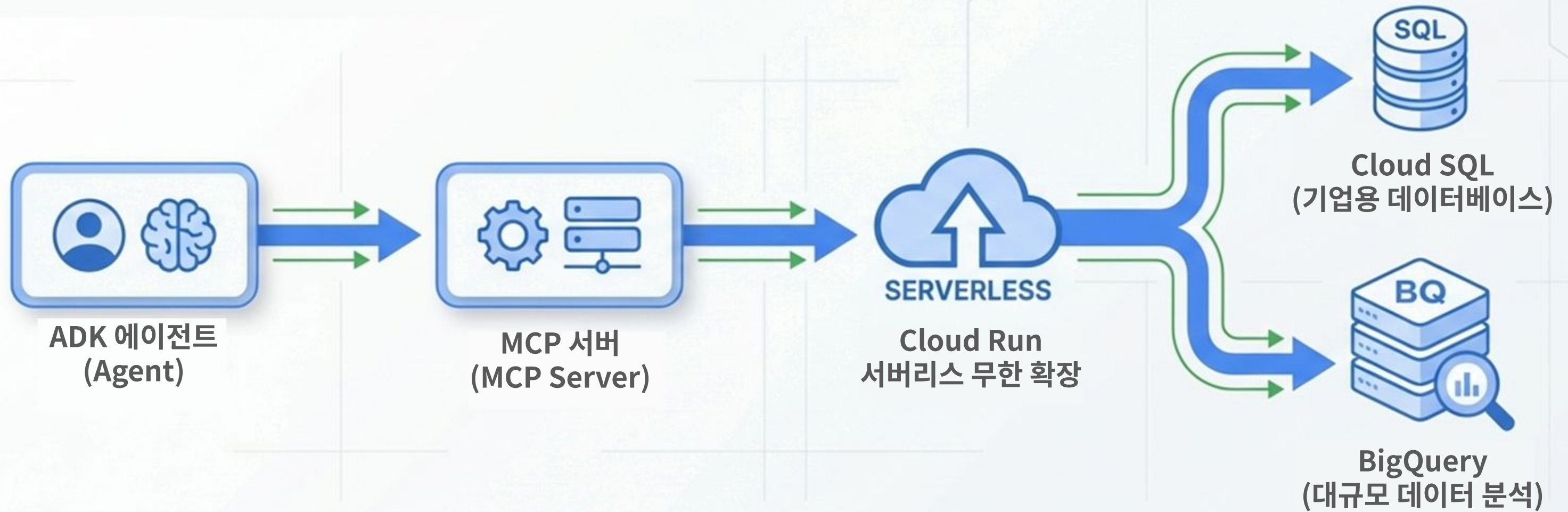
클라우드 생태계의 확장 : MCP 도입



MCP (Model Context Protocol) 는 에이전트의 내부 ‘두뇌’를 로컬 노트북 환경을 넘어 클라우드와 외부 세계라는 ‘팔, 다리’로 확장시키는 혁신적인 표준 프로토콜입니다.

파일 관리 시스템부터 외부 독립 서버 연동까지, 복잡한 커스텀 도구를 일일이 개발할 필요 없이 표준화된 단일 방식으로 외부 자원과 연결할 수 있습니다.

구글 클라우드 인프라와의 완벽한 엔터프라이즈 통합



엔터프라이즈 전환

단순한 로컬 스크립트를 넘어, 트래픽 폭증에도 견디는 엔터프라이즈 수준의 확장성과 안정성을 갖춘 아키텍처를 구현합니다.

Cloud Run 구동

컨테이너화된 MCP 서버를 Cloud Run 서버리스 환경에서 구동하여, 인프라 관리 없이 무한한 확장성을 확보합니다.

BigQuery 연동

에이전트가 방대한 기업 내부 데이터베이스에 직접 접근하고 SQL 쿼리를 실행하여, 완벽한 데이터 기반의 의사결정을 내립니다.

보안 인프라 : Service Account 인증



시스템 레벨 인증 (System-Level Auth)

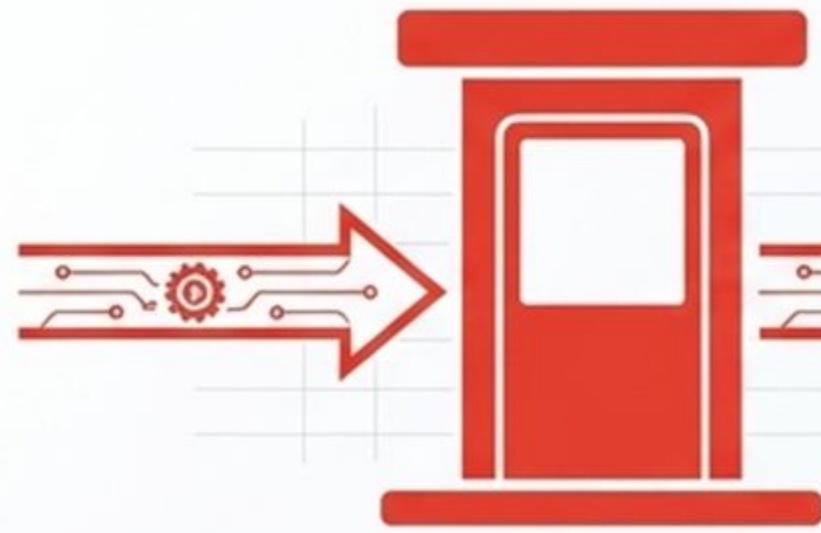
클라우드 자원에 대한 접근은 개별 사용자 계정이 아닌 철저히 통제된 시스템 레벨의 서비스 계정(Service Account)을 통한 인증을 구현해야 합니다.

내장된 보안 가드레일(Built-in Guardrails)

ADK는 보안 검증 흐름을 구현할 수 있는 구조를 제공합니다. 안전하게 획득한 자격 증명(Credential)은 session.state에 자동으로 캐싱되어 불필요한 재인증을 방지하고 성능을 극대화합니다.

생애주기 전반에 걸친 통제 : 콜백 가드레일

Agent Callback



전체 실행 흐름 통제

세션 상태의 특정 플래그를 감지하여 에이전트의 실행 자체를 사전에 차단하거나 최종 응답을 강제로 변경합니다.

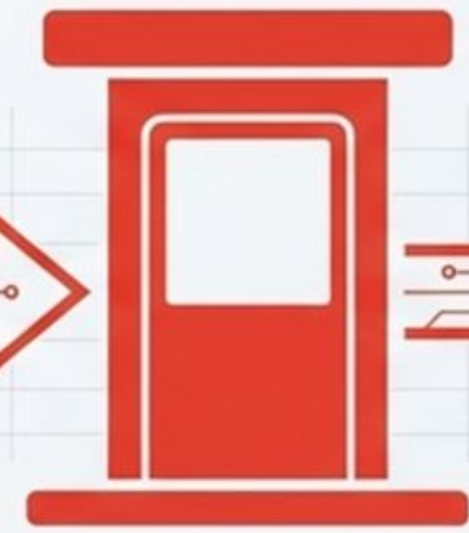
Model Callback



입출력 필터링 및 동적 수정

사용자의 입력을 가로채 PII(개인정보)나 욕설을 필터링하고, 상태 기반으로 시스템 프롬프트를 동적으로 수정합니다.

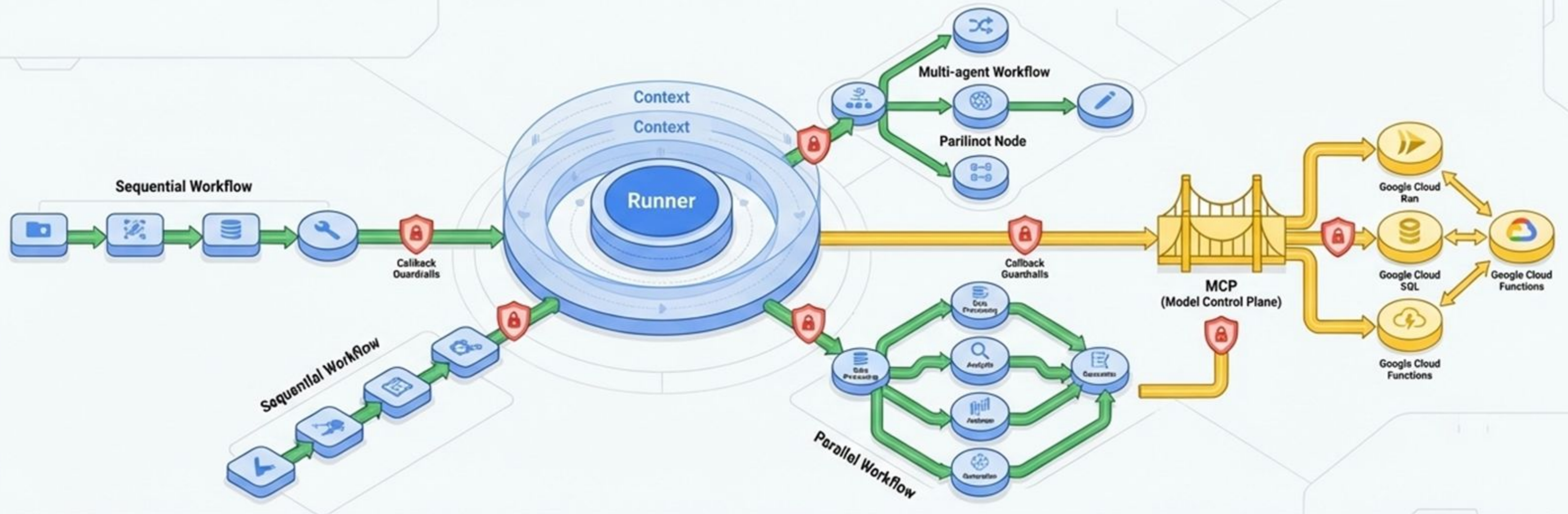
Tool Callback



인자 검증 및 장애 차단

LLM이 생성한 도구 실행 파라미터를 외부 시스템에 전달하기 전 사전 검증하여, 잘못된 인자로 인한 백엔드 장애를 원천 차단합니다.

자율적이고 안전한 확장형 AI 워크포스의 완성



단순한 프롬프트 엔지니어링의 시대는 이미 넘었습니다.

결정론적 워크플로우(Orchestration), 무한한 인프라 확장성(MCP & GCP 통합), 강력하고 틈 없는 보안 제어(Context & Callbacks)를 가진 ADK 프레임워크를 통해 비즈니스의 난제를 해결할 궁극적인 관제 센터를 직접 구축하십시오.



실습

<https://explore.qwiklabs.com/discovery>



참고 링크

1. 공식 허브 및 소통 채널

ADK 공식 홈페이지

<https://adk.dev>

ADK 프레임워크의 메인 허브로, 핵심 아키텍처 및 지원 언어, 최신 업데이트를 확인하는 시작점입니다.

ADK 글로벌 커뮤니티

<https://adk.dev/community/>

전 세계 빌더들의 소통 공간으로 기술 질문, 커뮤니티 콜 참여, 커스텀 기능 공유가 이루어집니다.

2. 문서 및 교육 가이드

ADK 한국어 공식 문서

<https://adk-labs.github.io/adk-docs/ko/>

국내 개발자를 위해 최적화된 기술 문서로 환경 셋업부터 아키텍처 가이드를 제공합니다.

Google Cloud ADK 공식 가이드

<https://docs.cloud.google.com/build/adk>

엔터프라이즈 환경의 AI 에이전트 빌드, 실행, 평가 및 확장 가이드를 포함합니다.

GEAR / Google Skills 학습 패스

<https://www.skills.google/paths/3545>

에이전트 메모리 관리, 도구 적용 등 가이드형 실습을 통해 공식 배지를 획득할 수 있습니다.

3. 오픈소스 및 배포처

ADK 공식 GitHub 저장소

<https://github.com/google/adk-web>

소스 코드와 시각적 디버깅 도구(CLI/Web UI)가 관리되는 오픈소스 저장소입니다.

언어별 패키지 관리자

```
pip install google-adk
npm install @google/adk
```

Python, TypeScript 등 환경에 따른 패키지 배포 및 버전 체크를 위한 공식 채널입니다.



QnA



수고하셨습니다.